CSE341 Assignment

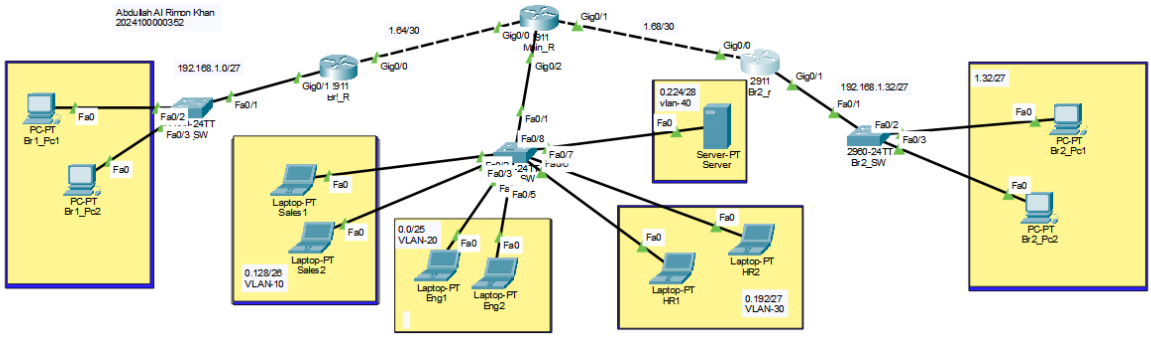
Part A: Network Design and Analysis

1. Network Design Proposal

1.1 Design Documentation

I designed a network for a medium sized company with a main office and two remote offices. The main office has Sales, Engineering and HR departments and a server room. I built it in Cisco Packet Tracer. Figure 1 shows the topology. Main_R is the main router. The branch routers Br1_R and Br2_R connect the two remote offices to it.

Figure 1: Network topology in Packet Tracer



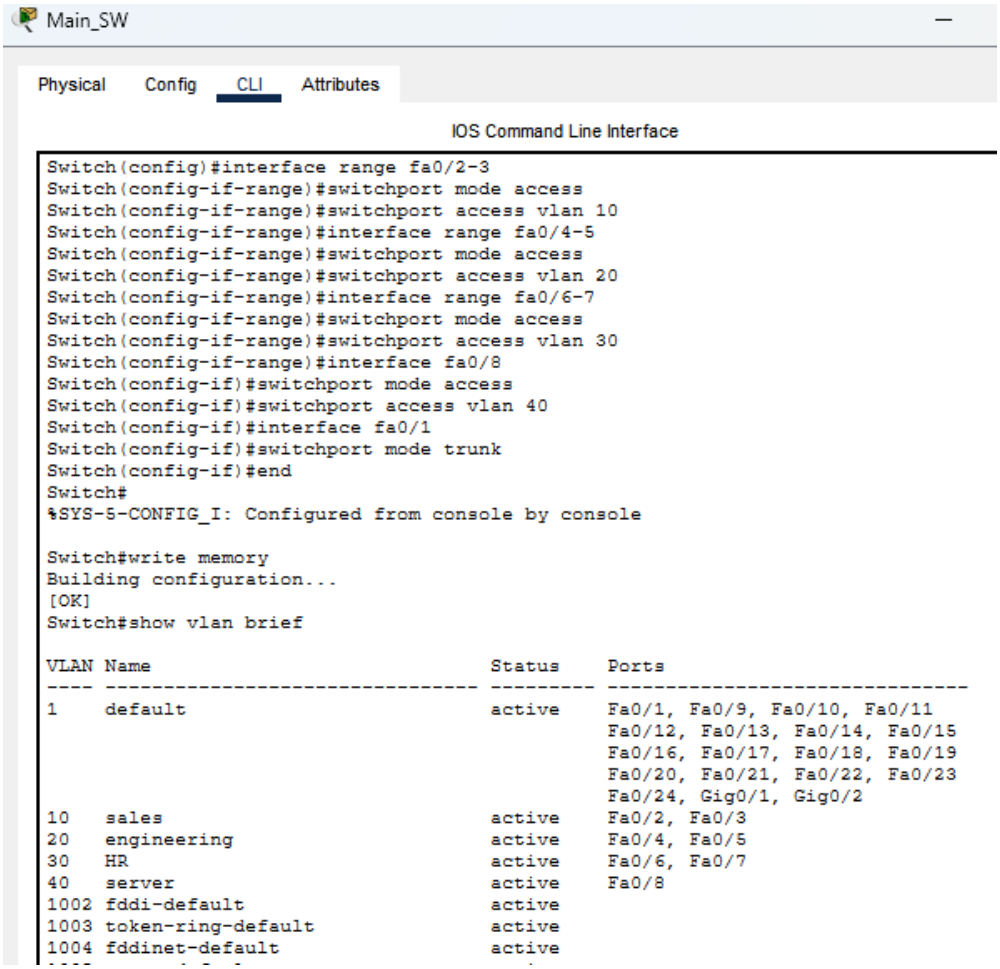
Devices used:

- 3 routers: Main_R, Br1_R, Br2_R (Cisco 2911)
- 3 switches: Main_SW, Br1_SW, Br2_SW (Cisco 2960-24TT)
- 1 server in the main office (DHCP and DNS)
- 6 laptops in the main office (Sales1, Sales2, Eng1, Eng2, HR1, HR2) and 4 PCs in the remote offices (Br1_Pc1, Br1_Pc2, Br2_Pc1, Br2_Pc2)

Connections: Main_R Gig0/0 goes to Br1_R Gig0/0, Main_R Gig0/1 goes to Br2_R Gig0/0, and Main_R Gig0/2 goes to Main_SW Fa0/1 (trunk port). Each branch router Gig0/1 goes to its own switch, and the PCs connect to the switch.

On Main_SW I made four VLANs: Sales (VLAN 10) on Fa0/2-3, Engineering (VLAN 20) on Fa0/4-5, HR (VLAN 30) on Fa0/6-7 and Server (VLAN 40) on Fa0/8. The commands are in Figure 2.

Figure 2: VLAN configuration on Main_SW



VLANs need a router to talk to each other, so I used router on a stick. Main_R uses one cable (Gig0/2) with four subinterfaces, and each one is the gateway of a VLAN (Figure 3).

Figure 3: Main_R interfaces (show ip interface brief)

```
[OK]
Main_R#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.65    YES manual up                    down
GigabitEthernet0/1     192.168.1.69    YES manual up                    down
GigabitEthernet0/2     unassigned      YES unset  up                    up
GigabitEthernet0/2.10  192.168.0.129   YES manual up                    up
GigabitEthernet0/2.20  192.168.0.1     YES manual up                    up
GigabitEthernet0/2.30  192.168.0.193   YES manual up                    up
GigabitEthernet0/2.40  192.168.0.225   YES manual up                    up
Vlan1                  unassigned      YES unset  administratively down down
Main_R#
```

Figure 4 is the configuration of Br1_R. It has an address for the link to Main_R and one for its LAN, and a default route to Main_R (192.168.1.65). Br2_R is set up the same way (Figure 5).

Figure 4: Configuration of Br1_R

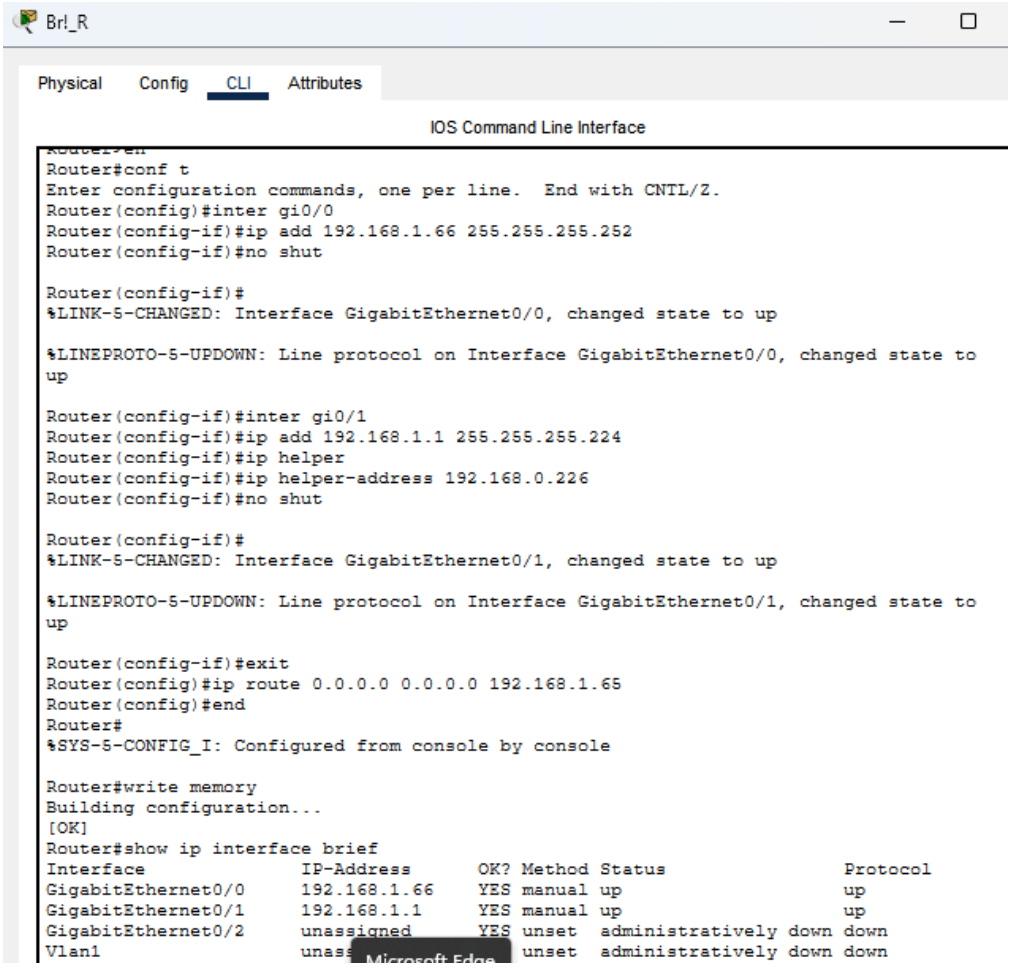
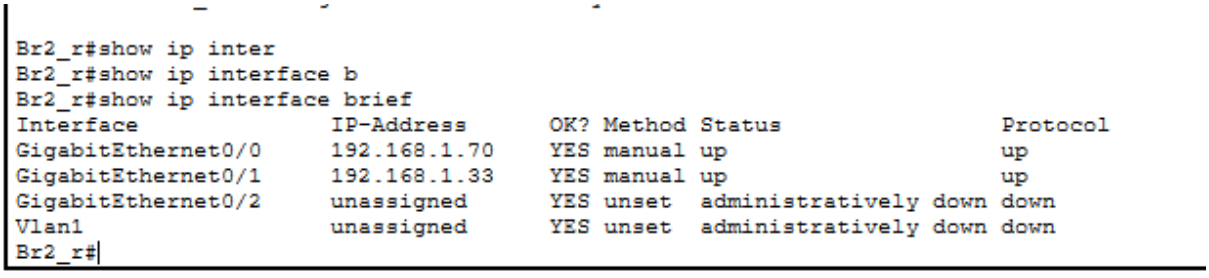


Figure 5: Br2_R interfaces (show ip interface brief)



1.2 Layered Architecture

Table 1: OSI layers in my network

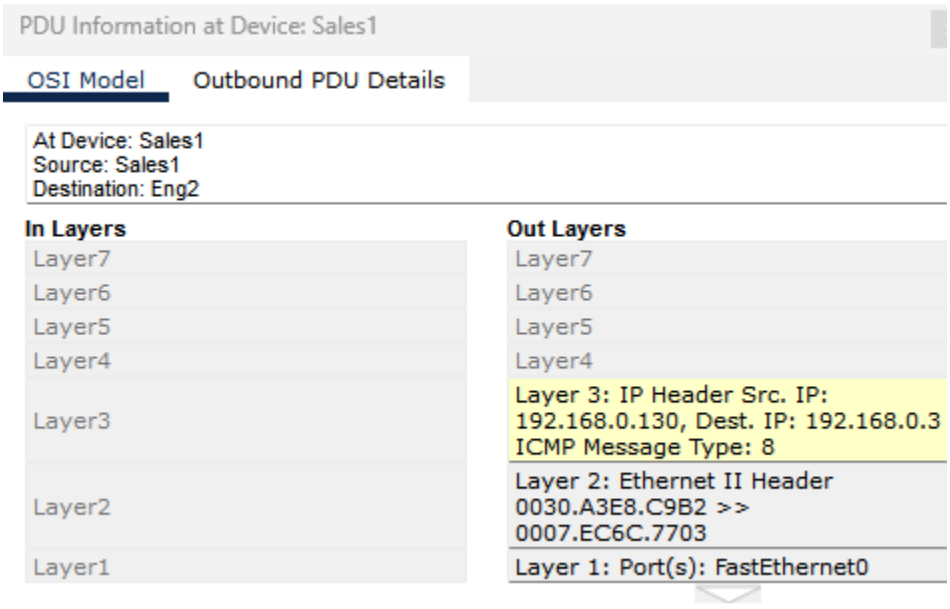
OSI layer	Function	In my network
7 Application	Network services for the user programs	DHCP and DNS on the server
6 Presentation	Data format, encoding, encryption	Not used much in my design
5 Session	Starts, keeps and ends connections	Not used much in my design
4 Transport	End to end delivery, ports, TCP or UDP	DHCP and DNS use UDP
3 Network	IP addressing and routing between networks	The 3 routers, IP plan, ICMP (ping)
2 Data Link	Frames, MAC addresses, VLANs	Switches, Ethernet frames, Dot1q trunk
1 Physical	Cables, ports, signals	Cables and ports (Fa0/1, Gig0/0)

Table 2: TCP/IP layers in my network

TCP/IP layer	OSI layers	In my network
Application	7, 6, 5	DHCP, DNS
Transport	4	UDP (TCP when needed)
Internet	3	IP, ICMP, routers
Network Access	2, 1	Ethernet, VLAN, switches, cables

To check the layers I sent a ping from Sales1 to Eng2 in simulation mode and looked at the PDU at each device. At Sales1 the ping is an ICMP echo request (type 8) from 192.168.0.130 to 192.168.0.3. Layer 2 adds an Ethernet header with the MAC of the gateway, because Eng2 is in another network.

Figure 6: Sales1 sending the ping (outbound PDU)



Main_R receives the frame on Gig0/2 with a Dot1q (VLAN) header, finds that the destination is in VLAN 20 and sends it out of the same port with a new MAC header. The IP addresses

stay the same, only the MAC addresses change. Eng2 then sends an echo reply (type 0) back the same way.

Figure 7: Main_R receiving and forwarding the request

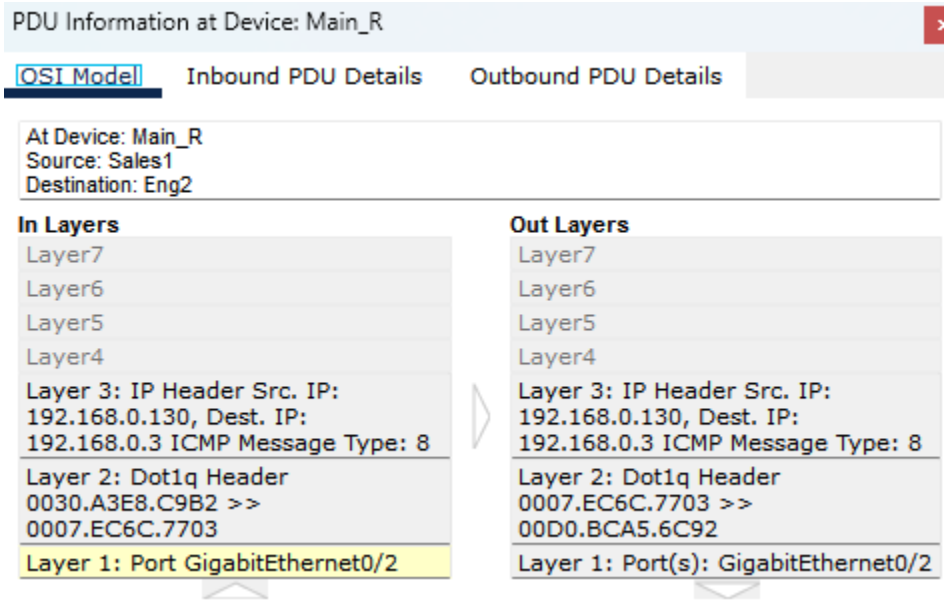


Figure 8: Eng2 receiving the request and sending the reply

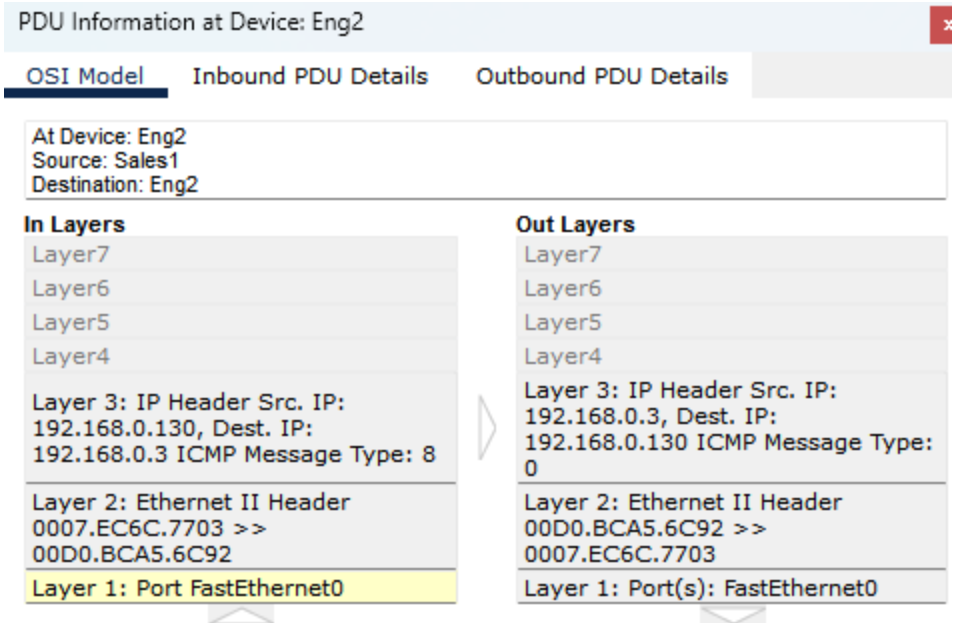


Figure 9: Main_R passing the reply back

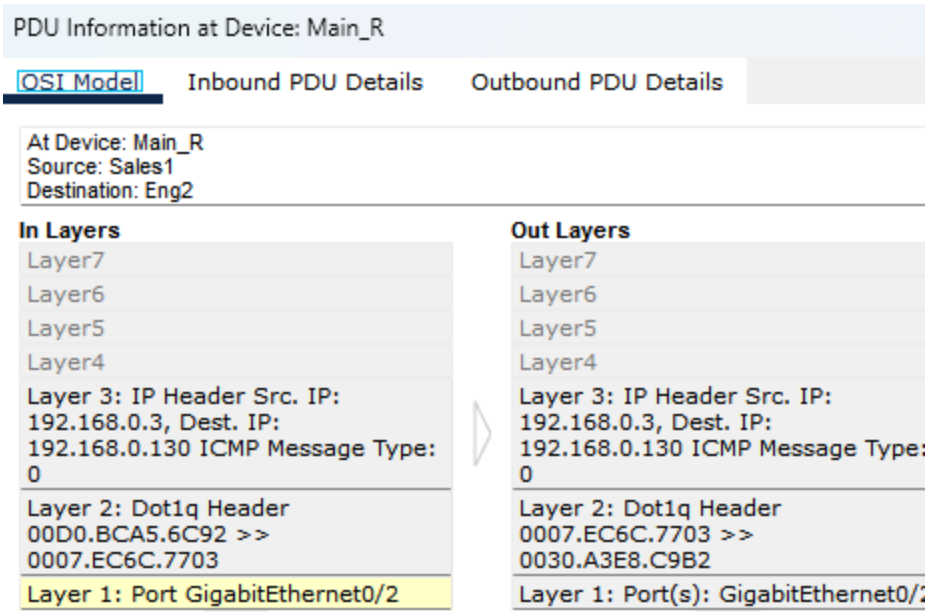
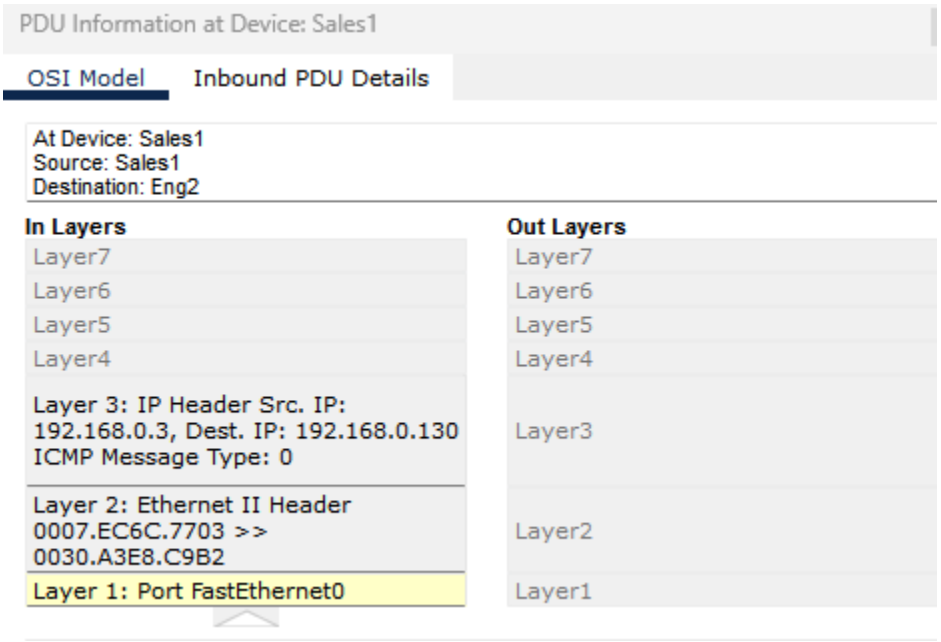


Figure 10: Sales1 receiving the reply (inbound PDU)



2. Addressing Scheme

2.1 Address Allocation

I used private addresses. The main office uses 192.168.0.0/24 divided between the four VLANs. The remote offices and the router links use 192.168.1.0/24. For physical addressing every device has a MAC address, for example Sales1 is 0030.A3E8.C9B2, Main_R is

0007.EC6C.7703 and Eng2 is 00D0.BCA5.6C92 (seen in the PDU figures).

Table 3: Subnet table

Network	Network address	Subnet mask	Gateway	Usable range	Broadcast
Engineering (VLAN 20)	192.168.0.0/25	255.255.255.128	192.168.0.1	192.168.0.1 - 192.168.0.126	192.168.0.127
Sales (VLAN 10)	192.168.0.128/26	255.255.255.192	192.168.0.129	192.168.0.129 - 192.168.0.190	192.168.0.191
HR (VLAN 30)	192.168.0.192/27	255.255.255.224	192.168.0.193	192.168.0.193 - 192.168.0.222	192.168.0.223
Server (VLAN 40)	192.168.0.224/28	255.255.255.240	192.168.0.225	192.168.0.225 - 192.168.0.238	192.168.0.239
Remote office 1	192.168.1.0/27	255.255.255.224	192.168.1.1	192.168.1.1 - 192.168.1.30	192.168.1.31
Remote office 2	192.168.1.32/27	255.255.255.224	192.168.1.33	192.168.1.33 - 192.168.1.62	192.168.1.63
Link Main_R - Br1_R	192.168.1.64/30	255.255.255.252	none	192.168.1.65 - 192.168.1.66	192.168.1.67
Link Main_R - Br2_R	192.168.1.68/30	255.255.255.252	none	192.168.1.69 - 192.168.1.70	192.168.1.71

Table 4: Router and server addresses

Device	Interface	IP address
Main_R	Gi0/0 (to Br1_R)	192.168.1.65 / 30
Main_R	Gi0/1 (to Br2_R)	192.168.1.69 / 30
Main_R	Gi0/2.10 / .20 / .30 / .40	192.168.0.129 / 192.168.0.1 / 192.168.0.193 / 192.168.0.225
Br1_R	Gi0/0 and Gi0/1	192.168.1.66 / 30 and 192.168.1.1 / 27
Br2_R	Gi0/0 and Gi0/1	192.168.1.70 / 30 and 192.168.1.33 / 27
Server	FastEthernet0 (static)	192.168.0.226 / 28

2.2 Subnetting

I used VLSM, starting from the biggest subnet. Usable hosts = 2^n - 2, where n is the number of host bits.

- Engineering, about 100 hosts: 2^7 = 128, 126 usable, so /25.
- Sales, about 50 hosts: 2^6 = 64, 62 usable, so /26.
- HR, about 20 hosts: 2^5 = 32, 30 usable, so /27.
- Server VLAN, few hosts: 2^4 = 16, 14 usable, so /28.
- Each remote office, about 20 hosts: /27.
- Router links, 2 hosts: 2^2 = 4, 2 usable, so /30.

In the main office 240 of 256 addresses are used (128 + 64 + 32 + 16), and 192.168.0.240/28 is free. In 192.168.1.0/24 everything after 192.168.1.71 is free for more offices.

The DHCP server gives addresses from these subnets (Figure 11). Figures 12 to 16 show that the devices got an address from their own subnet, with the correct gateway and the DNS server 192.168.0.226.

Figure 11: DHCP pools on the server

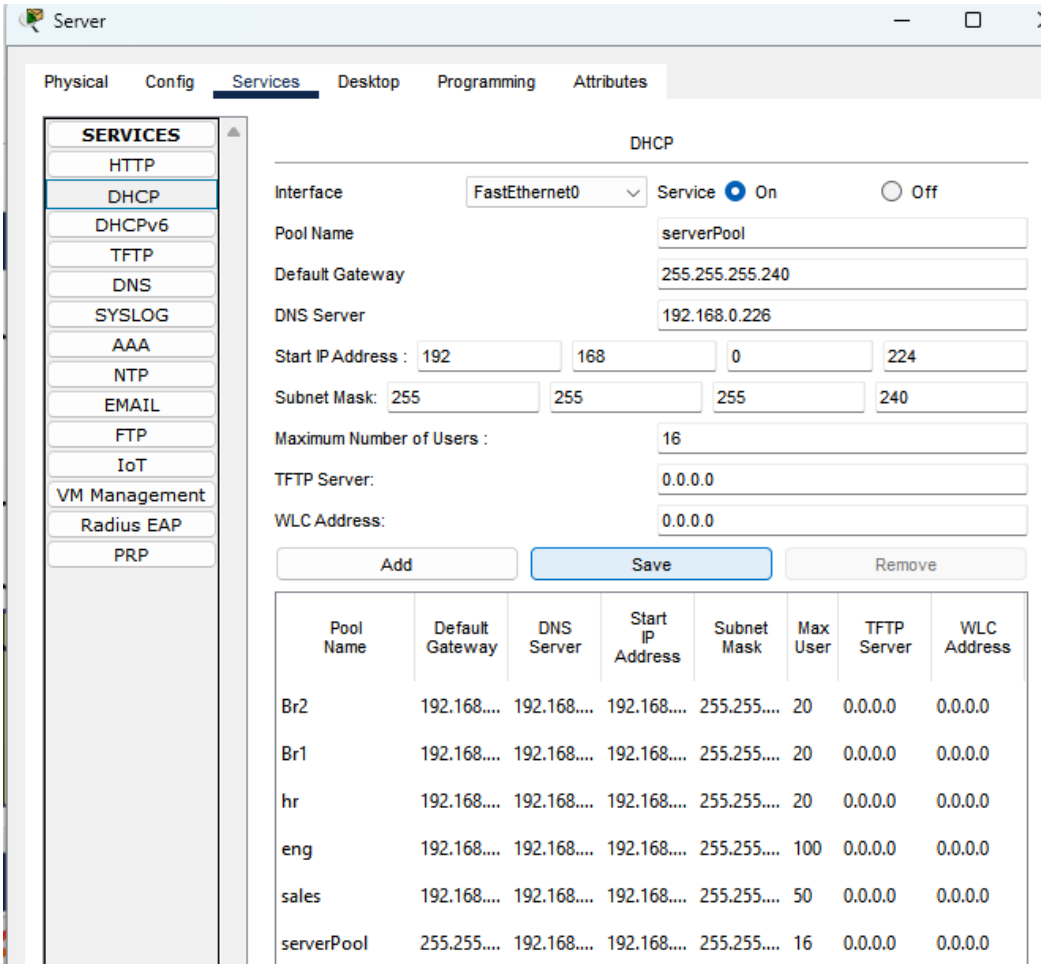


Figure 12: Br1_Pc1 got its address from DHCP

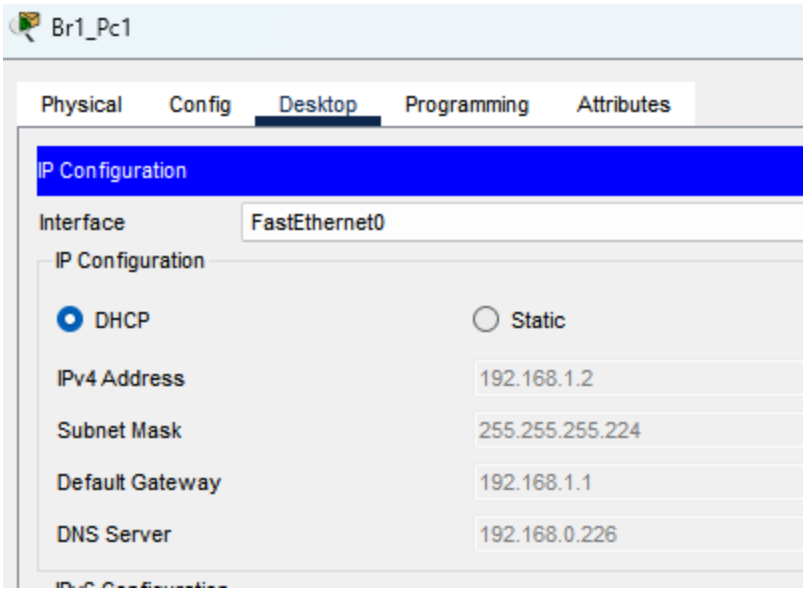


Figure 13: Sales1 got its address from DHCP

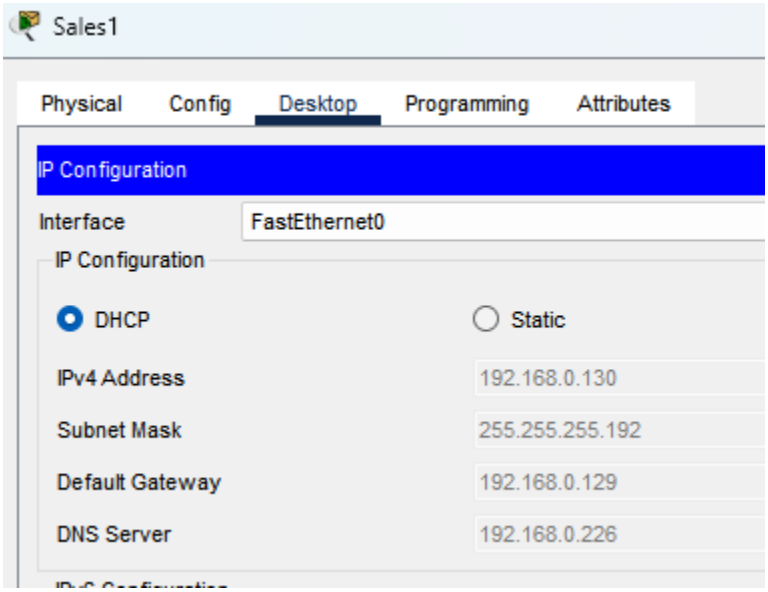


Figure 14: Eng1 got its address from DHCP

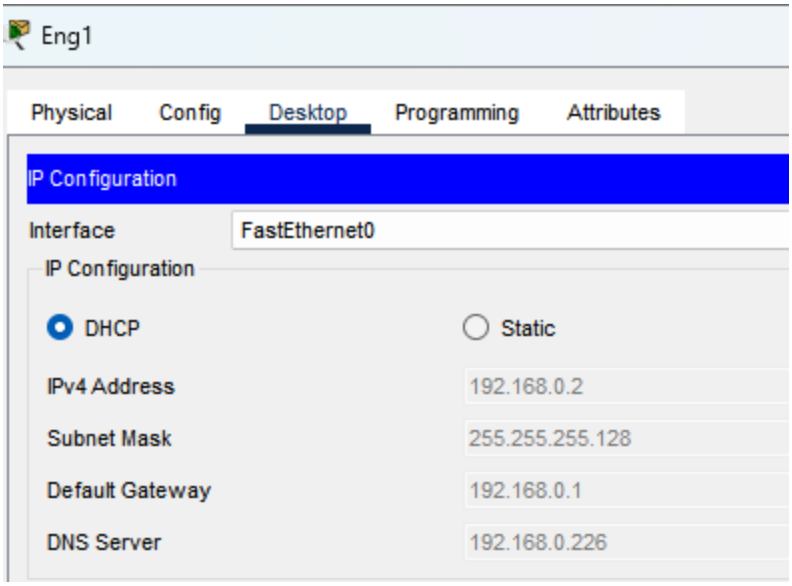


Figure 15: HR1 got its address from DHCP

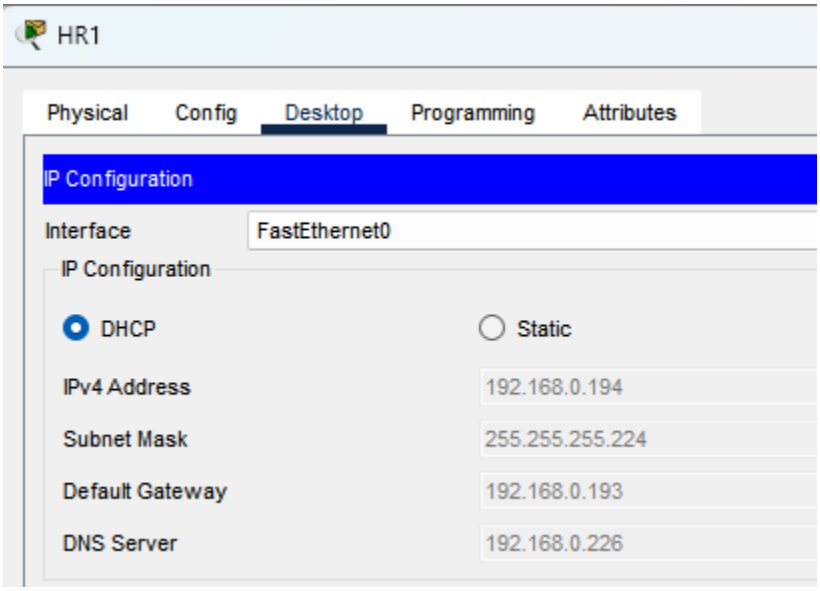
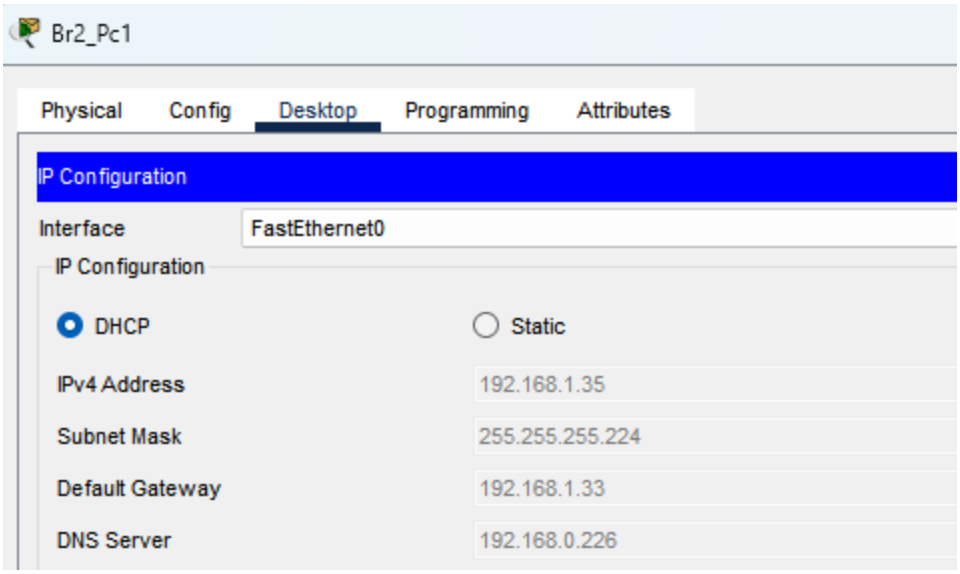


Figure 16: Br2_Pc1 got its address from DHCP



3. Protocol Selection and Justification

TCP/IP: It is the protocol suite of the internet. IP gives addresses and lets routers move packets between the offices. TCP is used when data must arrive correctly and UDP for small quick messages like DHCP and DNS. They work at the Internet and Transport layers.

DHCP: There are many devices, so setting IPs by hand takes time and can cause mistakes like duplicate addresses. DHCP does it automatically and also gives the gateway and DNS. The server is in VLAN 40 but the PCs are in other networks, and routers do not forward DHCP broadcasts. So I used ip helper-address 192.168.0.226 on the router interface (Figure 4). It works at the Application layer and uses UDP.

DNS: A name is easier to remember than a number. The server has the record www.company.com = 192.168.0.226 (Figure 18). DHCP gives this DNS server to every device. The server has a static IP (Figure 17) because other devices depend on its address. DNS is an Application layer protocol and uses UDP.

Figure 17: Static IP configuration of the server

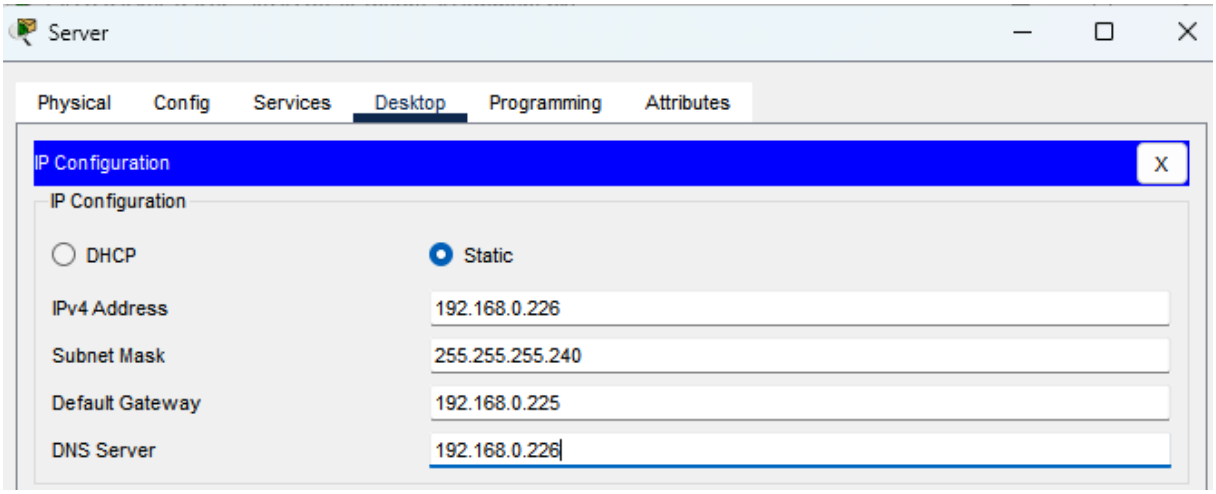
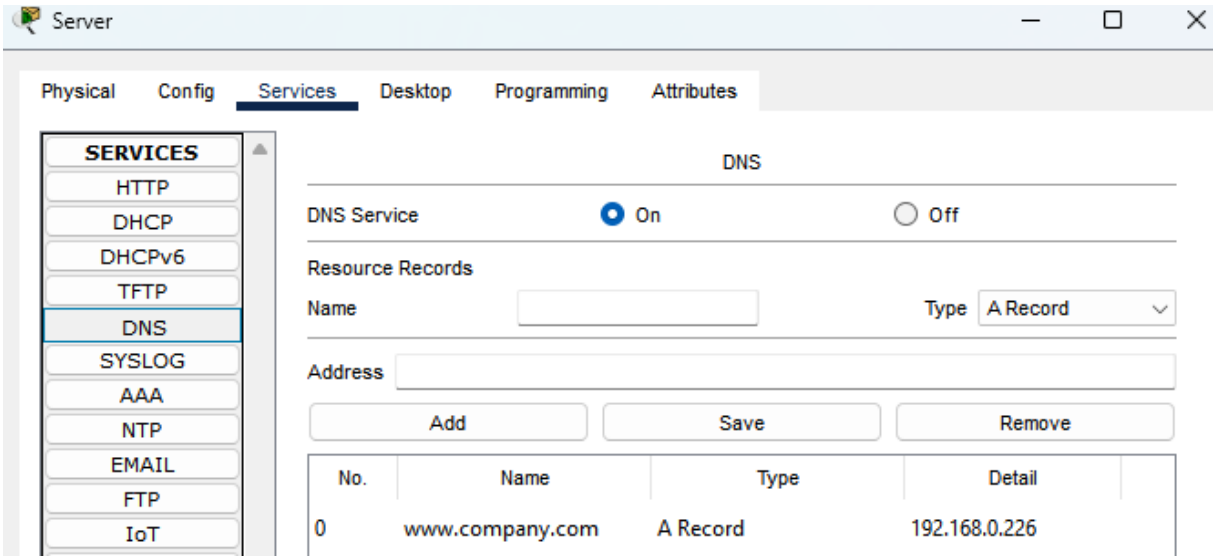


Figure 18: DNS record on the server



Others: ICMP for ping and tracert (Network layer), ARP to find a MAC from an IP, 802.1Q on the trunk to carry the VLANs (Data Link layer), and static routing with a default route on the branch routers, because the network is small and static routing is simple.

Testing: from Sales1 I pinged the server, Eng1 and www.company.com, and ran tracert to Br2_Pc1.

Figure 19: Ping from Sales1 to the server and to Eng1

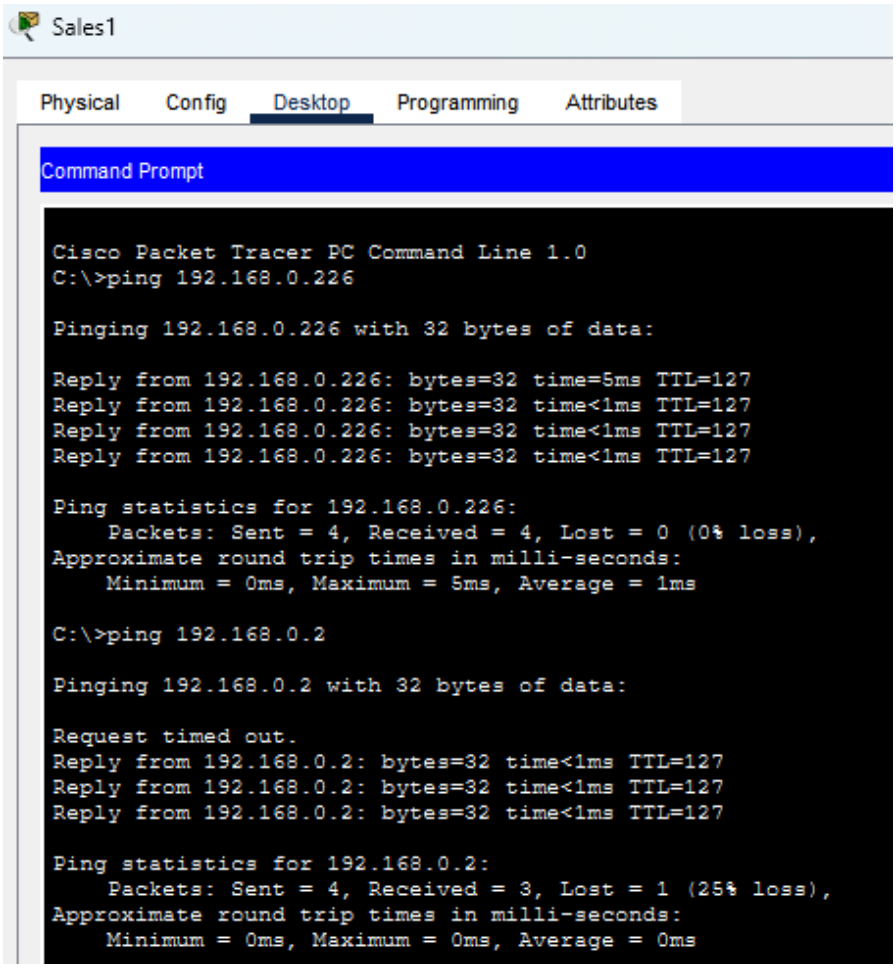
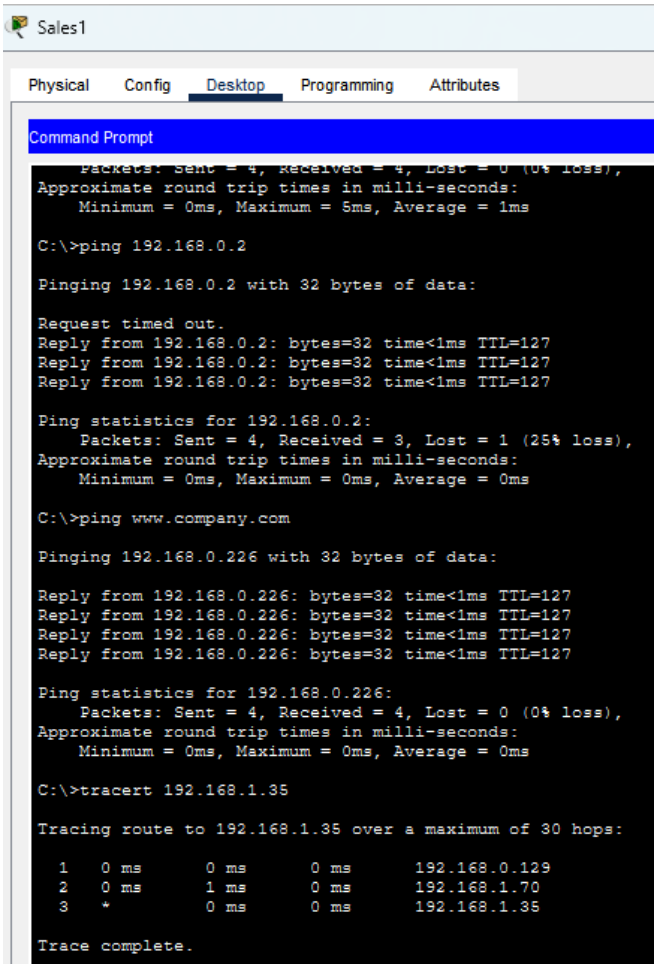


Figure 20: Ping to www.company.com and tracert to Br2_Pc1



All replies had TTL 127, which means one router (Main_R) is in between. The first ping to 192.168.0.2 timed out because the PC first needs ARP to find the MAC address, so the 25% loss is only that. The name www.company.com gave 192.168.0.226, so DNS works. The tracert shows Main_R (192.168.0.129), Br2_R (192.168.1.70) and then the PC 192.168.1.35.

4. Evaluation and Analysis

4.1 Performance Analysis

All replies came back in 0 to 5 ms, and only the first was slower because of ARP. Packet Tracer is a simulator so the times are not like a real network, but every path works. The address sizes fit the needs well: Engineering has 126 addresses for about 100 devices, Sales 62 for 50, HR 30 for 20 and each remote office 30 for 20. VLANs make the broadcast domains smaller and separate the departments. For reliability, the network works well when everything is up, but there is no backup if an important device or link fails.

4.2 Potential Issues and Improvements

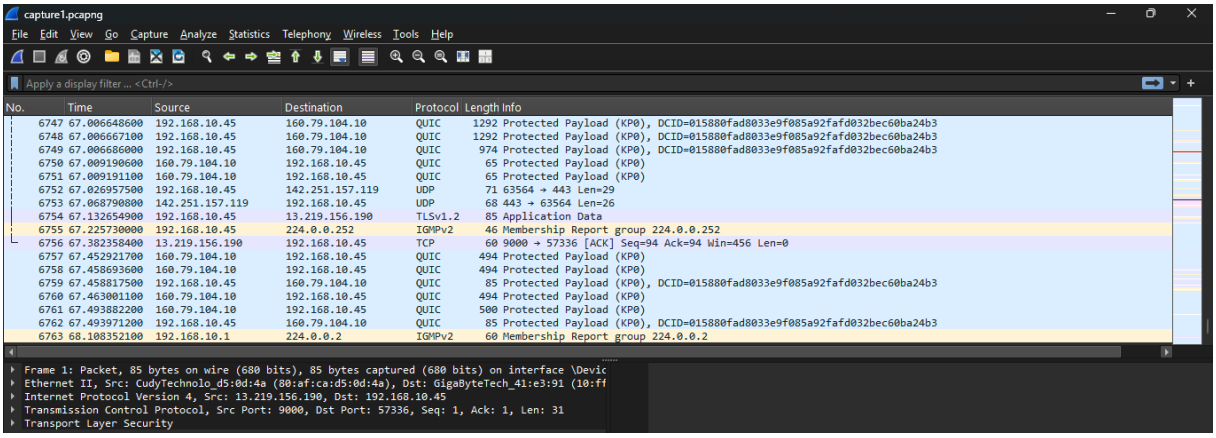
- Main_R is a single point of failure. Add a second router (for example with HSRP) and backup links.
- Router on a stick uses one cable for all inter-VLAN traffic and can get slow. Use a layer 3 switch.
- Static routes must be changed by hand for each new office. Use OSPF when the network grows.
- One DHCP/DNS server only. Add a backup server.
- No security. Use ACLs, port security, device passwords and shut the unused ports.
- HR (/27) and Server (/28) subnets are small. Use the free block 192.168.0.240/28 or change the sizes.

Part B: IP Protocol Analysis Using Wireshark

1. Part 1: Packet Capture Setup

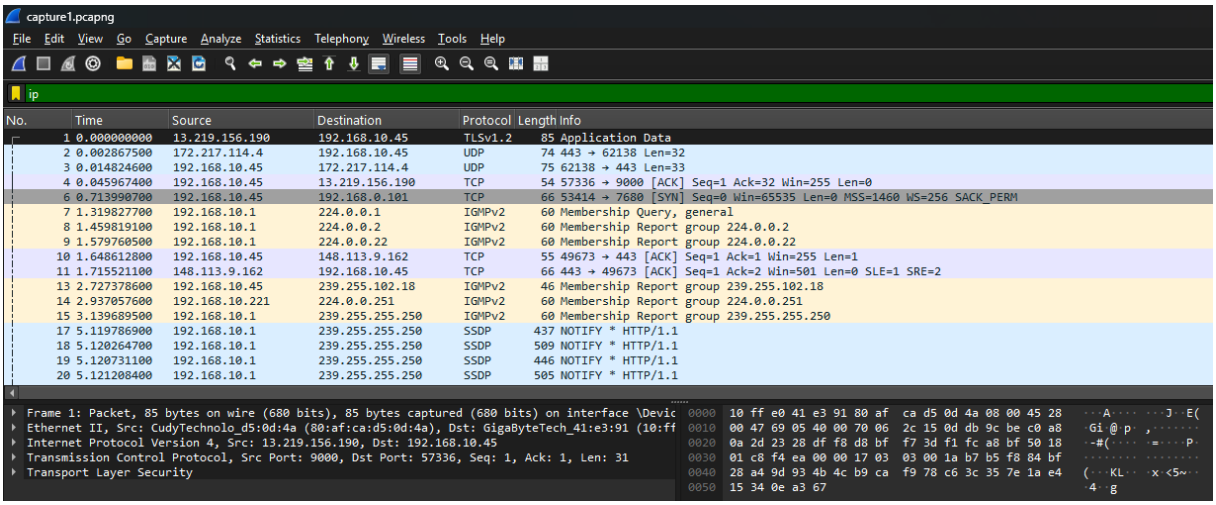
I installed Wireshark and started the capture on my network interface. My computer is 192.168.10.45 and the gateway is 192.168.10.1. I made traffic by browsing websites, pinging 8.8.8.8, running tracert to google.com and sending a large ping. The capture (capture1.pcapng) has thousands of packets, more than the 10 needed.

Figure 1: Packets in the capture (capture1.pcapng)



I used the filter ip to show only IP packets. The protocols seen were QUIC, TLSv1.2, TCP, UDP, IGMPv2 and SSDP.

Figure 2: Packets after applying the ip filter



2. Part 2: IP Packet Analysis

2.1 IP Header Fields

I selected packet 6090, an ICMP echo request from my computer to 8.8.8.8 (Figure 3).

Figure 3: IP header of packet 6090 (ping request to 8.8.8.8)

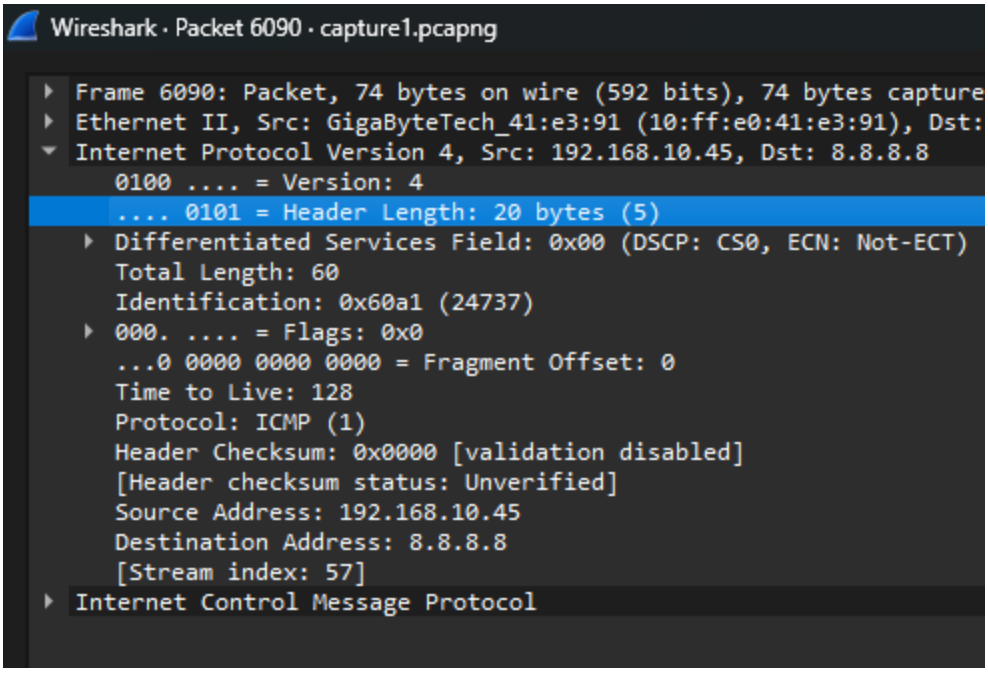


Table 1: IP header fields of packet 6090

Field	Value in my packet	Purpose
Source IP address	192.168.10.45	Address of the sender, used for the reply
Destination IP address	8.8.8.8	Address of the receiver, used by routers to forward the packet
Version	4	IP version, 4 means IPv4
Header Length	20 bytes (5)	Size of the IP header. 5 x 4 = 20 bytes, so no options
Total Length	60	Size of the whole packet: 20 bytes IP header + 40 bytes ICMP
Identification	0x60a1 (24737)	Number of the packet. Fragments of one packet share the same number
Time to Live (TTL)	128	Maximum number of routers the packet can pass. Stops endless loops
Protocol	ICMP (1)	Protocol inside the packet (1 = ICMP, 6 = TCP, 17 = UDP)
Header Checksum	0x0000 (validation disabled)	Checks the header for errors. Not verified here

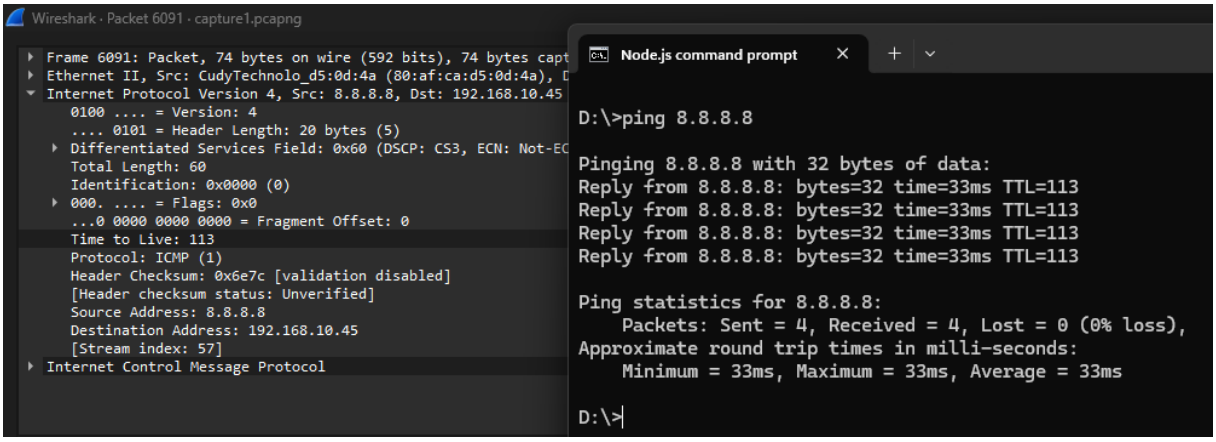
The frame is 74 bytes, which is 14 bytes Ethernet + 60 bytes IP, so it matches Total Length. Flags are 0x0 and offset is 0, so the packet is not fragmented.

2.2 Field Analysis

TTL: Figure 4 shows the reply from 8.8.8.8 (packet 6091). My request had TTL 128 and the reply came back with TTL 113. If the reply started from 128, it passed 15 routers (this is my guess, I don't know the start value of the server). TTL goes down by 1 at every router. This

stops packets from living forever in a loop. When it reaches 0 the router drops the packet and sends an ICMP Time Exceeded message back.

Figure 4: Ping reply from 8.8.8.8 (packet 6091) and the ping result



Protocol field: ICMP (1) came from ping and tracert. TCP (6) packets are in Figure 5, mostly Keep-Alive ACKs and data on port 443 (HTTPS). UDP (17) packets are in Figure 6, mostly QUIC on port 443 used by the browser.

Figure 5: TCP packets (filter: ip.proto == 6)

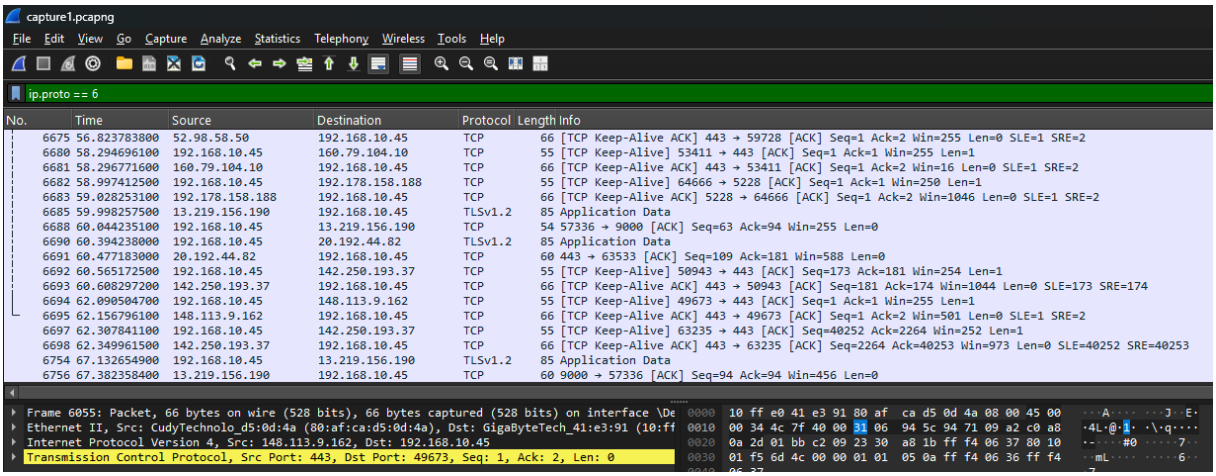
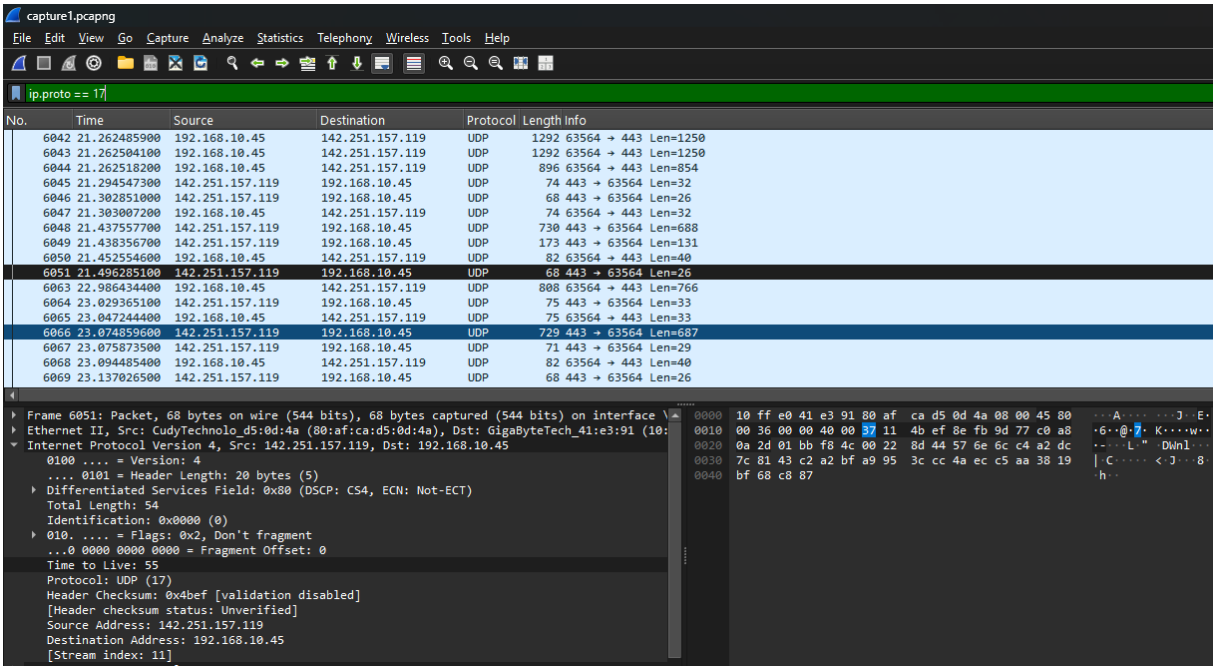


Figure 6: UDP packets (filter: ip.proto == 17)



3. Part 3: Routing Behaviour Analysis

3.1 Trace Route of a Packet

I ran tracert google.com and captured it in Wireshark. The path has 11 hops.

Figure 7: tracert google.com

```
D:\>tracert google.com

Tracing route to google.com [172.217.24.174]
over a maximum of 30 hops:

  1    <1 ms    <1 ms    <1 ms  192.168.10.1
  2     1 ms     1 ms     1 ms  172.16.100.58
  3     1 ms     1 ms     1 ms  172.16.16.217
  4     2 ms     1 ms     1 ms  10.60.80.25
  5     2 ms     1 ms     1 ms  10.60.80.90
  6     1 ms     1 ms     1 ms  10.60.80.82
  7     2 ms     2 ms     2 ms  10.161.189.89
  8     *        *        5 ms  10.162.191.29
  9    34 ms    34 ms    34 ms  10.161.145.5
 10    34 ms    34 ms    34 ms  142.251.200.20
 11    33 ms    33 ms    34 ms  lcmaaa-bc-in-f14.1e100.net [172.217.24.174]

Trace complete.
```

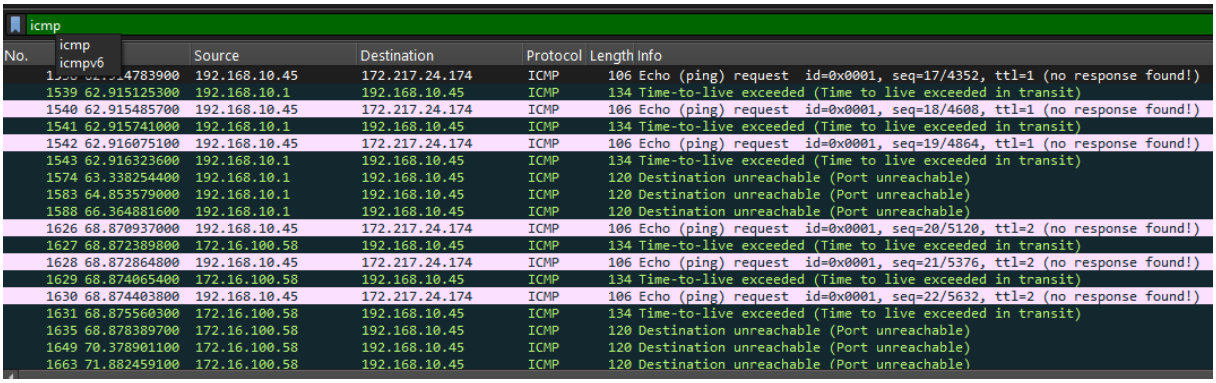
Table 2: Hops to google.com

Hop	IP address	Time	TTL of my packets
1	192.168.10.1	less than 1 ms	1
2	172.16.100.58	1 ms	2
3	172.16.16.217	1 ms	3
4	10.60.80.25	1 - 2 ms	4
5	10.60.80.90	1 - 2 ms	5
6	10.60.80.82	1 ms	6
7	10.161.189.89	2 ms	7
8	10.162.191.29	5 ms (two timeouts)	8
9	10.161.145.5	34 ms	9
10	142.251.200.20	34 ms	10
11	172.217.24.174 (google.com)	33 - 34 ms	11

Hop 1 is my gateway. The 172.16.x.x and 10.x.x.x addresses are private, so they are probably inside the ISP network. The delay is 1 to 2 ms for the first 7 hops and then jumps to about 34 ms at hop 9, maybe where a long distance link starts.

Tracert sends 3 packets with TTL 1, then 3 with TTL 2 and so on. The router where the TTL becomes 0 sends back Time Exceeded, and tracert reads its address from it. With the filter icmp (Figure 8), the requests with ttl=1 get a reply from 192.168.10.1 and the requests with ttl=2 from 172.16.100.58.

Figure 8: ICMP packets of tracert (filter: icmp)



3.2 Compare IP Header Changes

Figure 9: IP header of frame 1540 (TTL = 1)

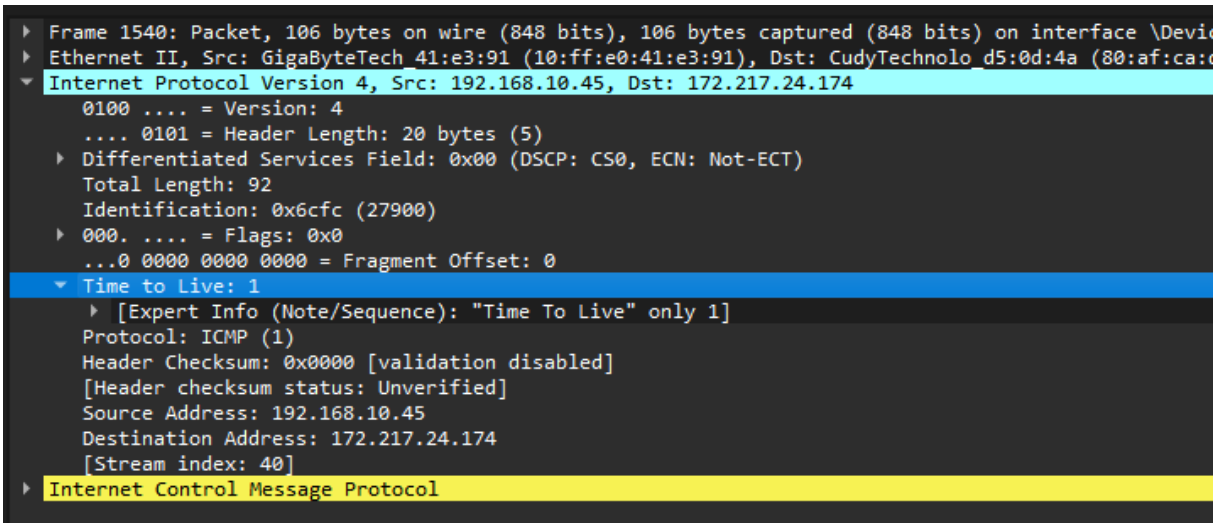
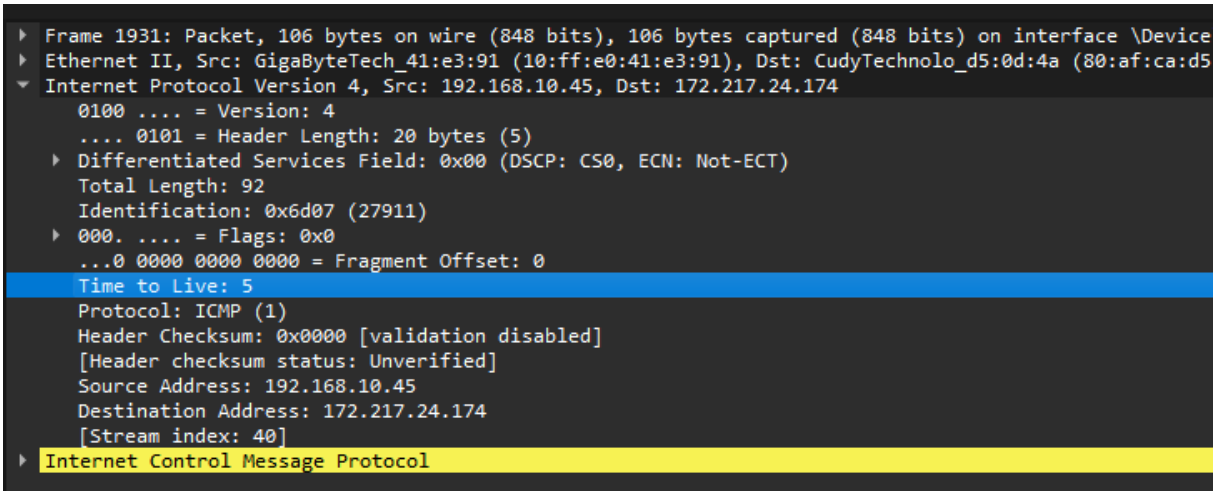


Figure 10: IP header of frame 1931 (TTL = 5)



Both packets are echo requests from 192.168.10.45 to 172.217.24.174. The addresses, version, header length, total length (92) and protocol are the same. Only the TTL (1 and 5) and the Identification (0x6cfc and 0x6d07) are different, because every new packet gets a new ID.

The checksum shows 0x0000 (validation disabled) in both. This is because of checksum offloading: the network card fills the checksum after Wireshark sees the packet, so I could not see it change hop by hop. Packets from outside have a real checksum, for example 0x6e7c in the ping reply. In theory every router lowers the TTL, so the header changes and the router must calculate the checksum again. The source and destination addresses do not change when passing routers (only NAT can change them).

4. Part 4: Error and Fragmentation Analysis

4.1 Packet Fragmentation

I used ping 8.8.8.8 -l 3000. The data is bigger than the Ethernet MTU of 1500 and I did not set the Don't Fragment flag. All 4 requests timed out (100% loss).

Figure 11: Large ping with 3000 bytes

```
D:\>ping 8.8.8.8 -l 3000

Pinging 8.8.8.8 with 3000 bytes of data:
Request timed out.
Request timed out.
Request timed out.
Request timed out.

Ping statistics for 8.8.8.8:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

With the filter ip.flags.mf == 1 || ip.frag_offset > 0 I could see the fragments. Each request was divided into 3 pieces. Wireshark shows two IPv4 fragments of 1514 bytes and then the full echo request in the third frame.

Figure 12: Fragmented packets (filter: ip.flags.mf == 1 || ip.frag_offset > 0)

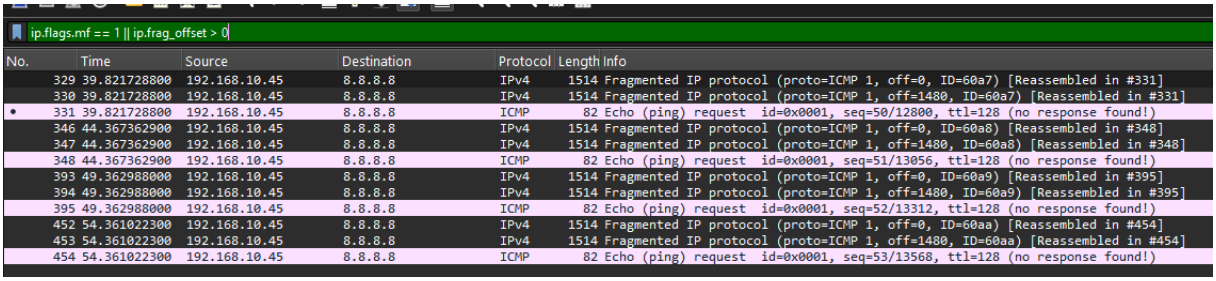


Figure 13 shows the first fragment (frame 329): Total Length 1500, Identification 0x60a7, More Fragments set, Don't Fragment not set, Fragment Offset 0, 1480 bytes of data, reassembled in frame 331.

Figure 13: First fragment, frame 329 (More Fragments flag set)

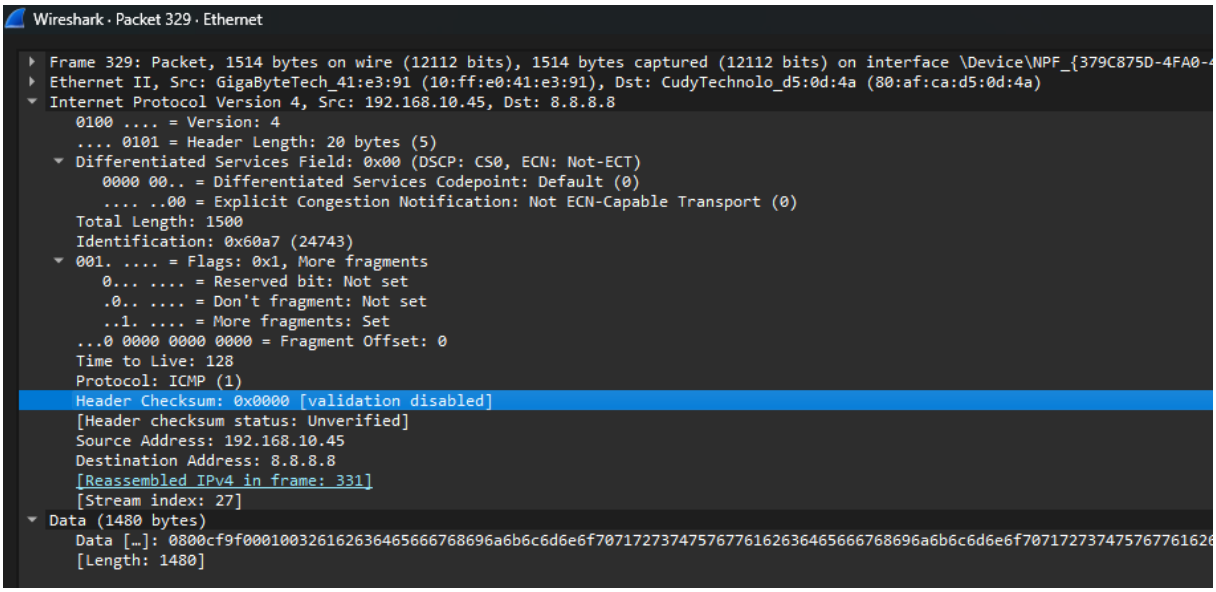


Table 3: Fragments of the first large ping

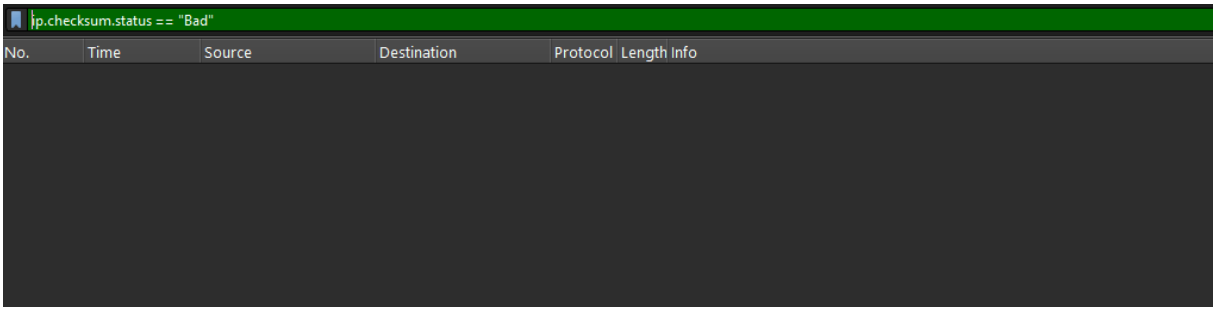
Fragment	Frame	Fragment offset	More Fragments	Data bytes
1	329	0	1 (set)	1480
2	330	1480 bytes (185 in the field)	1 (set)	1480
3	331	2960 bytes (370 in the field)	0 (last)	48

The data is 3000 + 8 bytes of ICMP header = 3008 bytes, and each fragment carries up to 1480 bytes. All three have the same Identification, source and destination, so the receiver knows they belong together. It puts them in order using the offset and knows the last one because More Fragments is 0. The next pings used the IDs 0x60a8, 0x60a9 and 0x60aa. There was no reply (Wireshark says no response found). I am not sure why, maybe the fragments were dropped or the server does not answer fragmented pings.

4.2 Checksum Errors

I used the filter ip.checksum.status == "Bad" and no packet was found.

Figure 14: Checksum filter result (no packets)



One reason is that checksum validation is disabled, so the status is Unverified. Another reason is that packets with a bad checksum are normally dropped by the network card or router before they reach the computer. Checksum errors can come from noise, bad cables or faulty devices. The IP checksum only covers the header. A packet with a wrong header checksum is dropped and IP does not tell the sender or send it again. TCP will send the data again after a timeout, but with UDP the data is lost.

5. Concluding Report

In this assignment I captured traffic on my own computer (192.168.10.45) with Wireshark and studied the IP packets. I made traffic by browsing, pinging 8.8.8.8, running tracert to google.com and sending a big ping.

First I studied the IP header. An IPv4 header is 20 bytes when there are no options. The fields I looked at were version, header length, total length, identification, TTL, protocol, checksum and the two IP addresses. The ping packet I selected had version 4, header length 20 bytes, total length 60 bytes, TTL 128 and protocol ICMP. The total length of 60 is 20 bytes of header plus 40 bytes of ICMP data, and with the 14 byte Ethernet header the frame is 74 bytes, which matches Wireshark. The protocol field tells the receiver what is inside, and I saw ICMP (1), TCP (6) and UDP (17). Most TCP packets were HTTPS traffic on port 443 and most UDP packets were QUIC.

The TTL field was the most interesting part. My ping request left with TTL 128 and the reply from 8.8.8.8 arrived with TTL 113, so if it started from 128 it passed 15 routers. Every router reduces the TTL by 1 and when it reaches 0 the packet is dropped and a Time Exceeded message goes back to the sender. This stops a packet from going round in a loop forever. Tracert uses the same idea. It sends packets with TTL 1, 2, 3 and so on, and every router that drops one shows its address. The path to google.com had 11 hops. The delay was about 1 or 2 ms for the first 7 hops but jumped to about 34 ms at hop 9.

When a router changes the TTL it must calculate the header checksum again. I could not see this because all my outgoing packets showed 0x0000 with validation disabled (checksum offloading). The filter for bad checksums also found nothing. In a real network a packet with a wrong checksum is dropped and IP does not tell the sender. TCP sends the data again after a timeout, but a UDP application just loses it.

For fragmentation I used ping 8.8.8.8 -l 3000. It is bigger than the MTU of 1500, so each request was split into three fragments with the same identification (0x60a7 for the first ping), offsets 0, 1480 and 2960, and More Fragments set on the first two. Wireshark showed them as reassembled in frame 331. The ping got no reply, so there was 100% loss. I am not sure about the exact reason. It shows one weakness of fragmentation: if one fragment is lost, the whole packet is lost.

In conclusion, every header field has a job. The addresses decide where the packet goes and the routers use the destination address for routing decisions. TTL prevents loops, the protocol

field tells which upper layer gets the data, identification and offset help in fragmentation and the checksum protects the header. For performance, more hops means more delay and fragmentation gives extra work to routers and hosts. For reliability, IP itself only does best effort delivery. It drops bad or expired packets and does not resend them, so TCP or the application must handle the lost data. Doing this assignment helped me understand how a simple ping or a website visit uses all of these fields at the same time, and why a small field like TTL is so important for the network to work properly.